Code Starts:

```
import pandas as pd
import requests
import numpy as np
import json


def get_professor_reviews(name):
    base_url = "https://api.planetterp.com/v1/professor"

    try:
        params = {
            'name': name,
            'reviews': 'true'
        }

        response = requests.get(base_url, params=params)

        if response.status_code != 200:
            print(f"Error {response.status_code} fetching professor {n
            return []

        data = response.json()

        reviews = []
        if 'reviews' in data and data['reviews']:
            for review in data['reviews']:
                reviews.append({
                    'professor': name,
                    'text': review['review'],
                    'stars': review['rating'],
                    'course': review.get('course', ''),
                    'date': review.get('created', '')
                })

        return reviews

    except Exception as e:
        print(f"Error processing {name}: {e}")
        return []
```

```
def collect_all_reviews():
    # the list of professors I want to analyze
    professors = [
        "Christopher Kauffman",
        "Justin Wyss-Gallifent",
        "Ilchul Yoon",
        "Barbara Michelato",
        "Steven Chadwick",
        "Mohammad Nayeem Teli"
    ]
    all_reviews = []
    for professor in professors:
        reviews = get_professor_reviews(professor)
        all_reviews.extend(reviews)

    df = pd.DataFrame(all_reviews)
    print(f"\nTotal reviews collected: {len(df)}")
    return df



# to collect data
df = collect_all_reviews()
```

```
Total reviews collected: 1041
```

```
import torch
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW
from transformers import DistilBertTokenizer, DistilBertForSequenceCl
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, accuracy_score, confu
from tqdm import tqdm
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# to use my cpu instead
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

```
Using device: cpu
```

```
class ProfessorReviewDataset(Dataset):

    def __init__(self, review_texts, stsr_labels, tokenizer, max_leng
        self.review_texts = review_texts
        self.stsr_labels = stsr_labels
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.review_texts)

    def __getitem__(self, idx):
        review_texts = str(self.review_texts[idx])
        stsr_labels = self.stsr_labels[idx]

        # tokenizes the text
        encoding = self.tokenizer(
            review_texts,
            truncation=True,
            padding='max_length',
            max_length=self.max_length,
            return_tensors='pt'
        )

        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(stsr_labels - 1, dtype=torch.long)
        }
```

```
def prepare_datasets(df, model_name='distilbert-base-uncased', test_s

    # to split the data
    train_texts, validation_texts, train_labels, validation_labels =
        df['text'].tolist(),
        df['stars'].tolist(),
        test_size=test_size,
        random_state=42,
        stratify=df['stars']
    )

    print(f"Training samples: {len(train_texts)}")
    print(f"Validation samples: {len(validation_texts)}")
```

```
    # initializing DistilBERT tokenizer and model
    tokenizer = DistilBertTokenizer.from_pretrained(model_name)
    model = DistilBertForSequenceClassification.from_pretrained(
        model_name,
        # to represent the 5 star categories
        num_labels=5
    )
    model.to(device)

    # spliting data into seperate training and validation datasets
    train_dataset = ProfessorReviewDataset(train_texts, train_labels,
    validation_dataset = ProfessorReviewDataset(validation_texts, val

    # prepares the dtaloaders dataloaders
    batch_size = 16
    train_loader = DataLoader(train_dataset, batch_size=batch_size, sl
    validation_loader = DataLoader(validation_dataset, batch_size=bat

    return model, tokenizer, train_loader, validation_loader, train_da

# calling the prepare data function
model, tokenizer, train_loader, validation_loader, train_dataset, val
```

```
Training samples: 832
Validation samples: 209
Warning: You are sending unauthenticated requests to the HF Hub. Please
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenti
```

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.04kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 2.73MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 5.11MB/s]

config.json: 100% 483/483 [00:00<00:00, 23.0kB/s]

model.safetensors: reconstructing file: 100% 268MB / 268MB, 21.9MB/s

model.safetensors: downloading bytes: 250MB, 23.1MB/s

Loading weights: 100% 100/100 [00:00<00:00, 3239.45it/s]

```
[transformers] DistilBertForSequenceClassification LOAD REPORT from: di
Key                      | Status     |
-------------------------+------------+-
vocab_layer_norm.bias    | UNEXPECTED |
vocab_transform.weight   | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
```

```
vocab_transform.bias    | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
classifier.bias         | MISSING    |
pre_classifier.bias     | MISSING    |
pre_classifier.weight   | MISSING    |
classifier.weight       | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/archite
- MISSING:      those params were newly initialized because missing fr
```

```
def setup_training(model, train_loader, epochs):

    # setting up the Optimizer
    optimizer = AdamW(model.parameters(), lr=2e-5)

    # setting up the Scheduler
    total_steps = len(train_loader) * epochs
    scheduler = get_linear_schedule_with_warmup(
        optimizer,
        num_warmup_steps=0,
        num_training_steps=total_steps
    )

    # using cross entropy loss function
    loss_fn = torch.nn.CrossEntropyLoss()

    return optimizer, scheduler, loss_fn, epochs

# setting up training
optimizer, scheduler, loss_fn, epochs = setup_training(model, train_l
```

```
# training one epoch
def train_epoch(model, data_loader, loss_fn, optimizer, device, schedu

    model = model.train()
    losses = []
    correct_predictions = 0

    for batch in tqdm(data_loader, desc="Training"):
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)
```

```
            outputs = model(input_ids=input_ids, attention_mask=attention_
            _, preds = torch.max(outputs.logits, dim=1)
            loss = loss_fn(outputs.logits, labels)

            correct_predictions += torch.sum(preds == labels)
            losses.append(loss.item())

            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.
            optimizer.step()
            scheduler.step()
            optimizer.zero_grad()

        return correct_predictions.double() / n_examples, np.mean(losses)

    # to evaluate the model's performance
    def eval_model(model, data_loader, loss_fn, device, n_examples):

        model = model.eval()
        losses = []
        correct_predictions = 0
        all_preds = []
        all_labels = []

        with torch.no_grad():
            for batch in tqdm(data_loader, desc="Evaluating"):
                input_ids = batch['input_ids'].to(device)
                attention_mask = batch['attention_mask'].to(device)
                labels = batch['labels'].to(device)

                outputs = model(input_ids=input_ids, attention_mask=atten
                _, preds = torch.max(outputs.logits, dim=1)
                loss = loss_fn(outputs.logits, labels)

                correct_predictions += torch.sum(preds == labels)
                losses.append(loss.item())
                all_preds.extend(preds.cpu().numpy())
                all_labels.extend(labels.cpu().numpy())

        # to convert predictions from 0-4 to 1 - 5.
        all_preds = [p + 1 for p in all_preds]
        all_labels = [l + 1 for l in all_labels]

        accuracy = correct_predictions.double() / n_examples
        rmse = np.sqrt(mean_squared_error(all_labels, all_preds))
```

```
    return accuracy, np.mean(losses), rmse, all_preds, all_labels
```

```
def train_model(model, train_loader, validation_loader, loss_fn, optimi
                device, train_dataset, validation_dataset, epochs=5):


    epoch_metrics = {
        'train_acc': [], 'train_loss': [],
        'validation_acc': [], 'validation_loss': [], 'validation_rmse':
    }

    best_accuracy = 0
    best_model_state = None

    for epoch in range(epochs):
        print(f'\nEpoch {epoch + 1}/{epochs}')

        # Train
        train_act, train_loss = train_epoch(
            model, train_loader, loss_fn, optimizer, device, scheduler,
        )

        # Evaluate
        validation_act, validation_loss, validation_rmse, validation_pr
            model, validation_loader, loss_fn, device, len(validation_d
        )

        # add to the metrics
        epoch_metrics['train_acc'].append(train_act.item())
        epoch_metrics['train_loss'].append(train_loss)
        epoch_metrics['validation_acc'].append(validation_act.item())
        epoch_metrics['validation_loss'].append(validation_loss)
        epoch_metrics['validation_rmse'].append(validation_rmse)

        print(f'Train loss: {train_loss:.4f}, Train acc: {train_act:.4f
        print(f'Validation loss: {validation_loss:.4f}, Validation acc:

        # save best model
        if validation_act > best_accuracy:
            best_accuracy = validation_act
            best_model_state = model.state_dict().copy()
            torch.save(best_model_state, 'best_model.pt')
            print("Saved best model!")
```

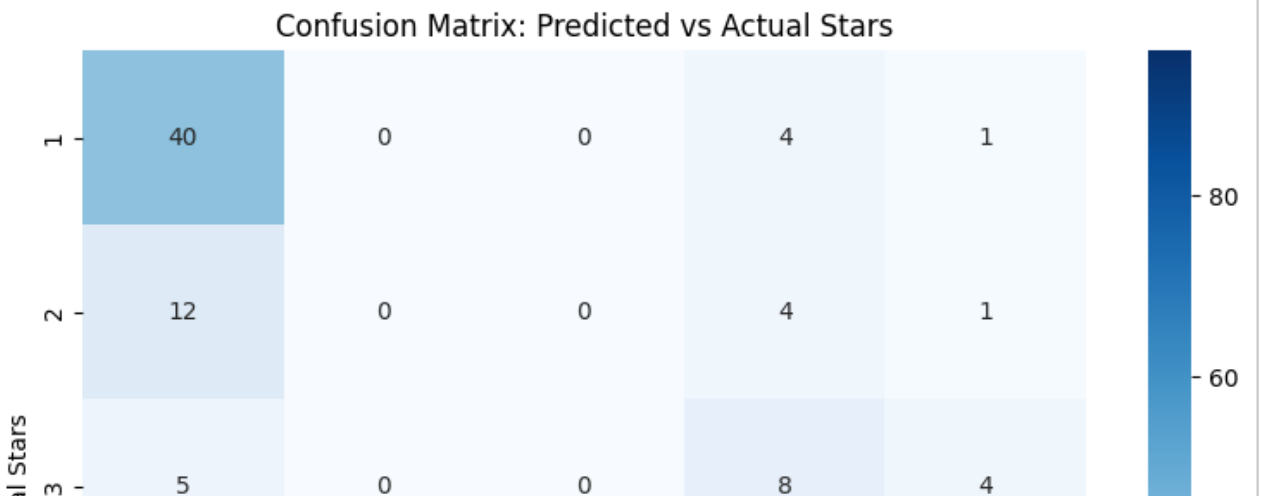
```
    # load best model
    model.load_state_dict(torch.load('best_model.pt'))

    return model, epoch_metrics, validation_preds, validation_labels

model, epoch_metrics, validation_preds, validation_labels = train_model
    model, train_loader, validation_loader, loss_fn, optimizer, schedul
    device, train_dataset, validation_dataset, epochs
)
```

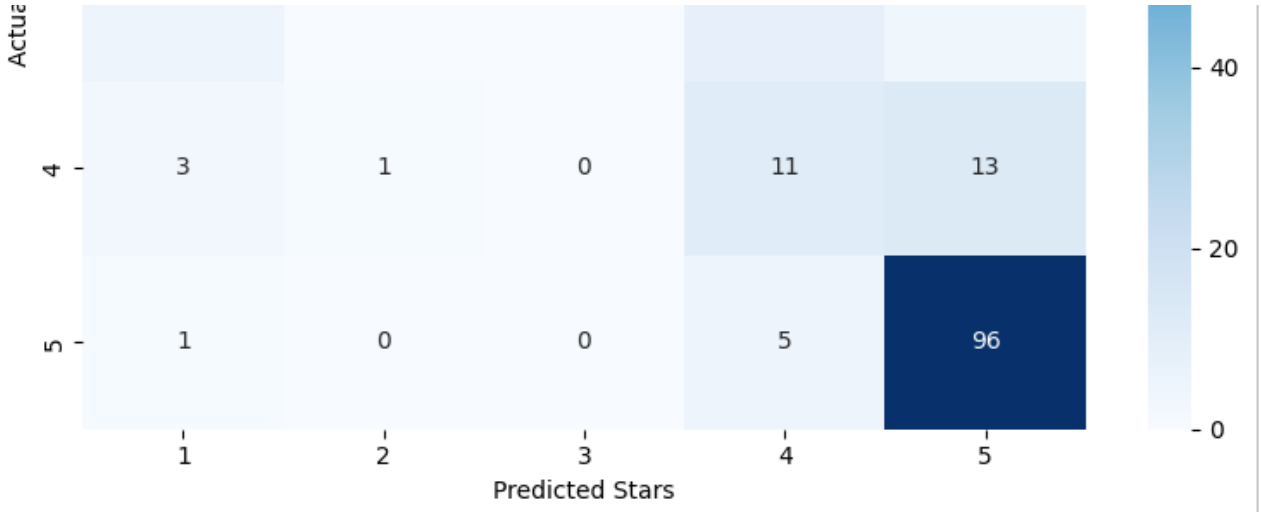
```
Epoch 1/5
Training: 100%|██████████| 52/52 [10:47<00:00, 12.45s/it]
Evaluating: 100%|██████████| 14/14 [00:48<00:00,  3.46s/it]
Train loss: 1.3613, Train acc: 0.4856
Validation loss: 1.1332, Validation acc: 0.5598, Validation RMSE: 1.85
Saved best model!

Epoch 2/5
Training: 100%|██████████| 52/52 [10:35<00:00, 12.21s/it]
Evaluating: 100%|██████████| 14/14 [00:47<00:00,  3.36s/it]
Train loss: 1.0341, Train acc: 0.6514
Validation loss: 0.8724, Validation acc: 0.6603, Validation RMSE: 1.23
Saved best model!

Epoch 3/5
Training: 100%|██████████| 52/52 [10:31<00:00, 12.15s/it]
Evaluating: 100%|██████████| 14/14 [00:47<00:00,  3.41s/it]
Train loss: 0.8048, Train acc: 0.7127
Validation loss: 0.7807, Validation acc: 0.6842, Validation RMSE: 1.10
Saved best model!

Epoch 4/5
Training: 100%|██████████| 52/52 [10:28<00:00, 12.08s/it]
Evaluating: 100%|██████████| 14/14 [00:47<00:00,  3.41s/it]
Train loss: 0.6882, Train acc: 0.7476
Validation loss: 0.7449, Validation acc: 0.7177, Validation RMSE: 0.93
Saved best model!

Epoch 5/5
Training: 100%|██████████| 52/52 [10:25<00:00, 12.02s/it]
Evaluating: 100%|██████████| 14/14 [00:46<00:00,  3.35s/it]Train loss:
Validation loss: 0.7290, Validation acc: 0.7033, Validation RMSE: 0.97
```

```
def accuracy_calculaton(predictions, act_stars):
```

```
    return accuracy_score(act_stars, predictions)

def plot_confusion_matrix(predictions, act_stars):

    cm = confusion_matrix(validation_labels, predictions, labels=[1,2,

    plt.figure(figsize=(8,6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=[1,2,3,4,5], yticklabels=[1,2,3,4,5])
    plt.title('Confusion Matrix: Predicted vs Actual Stars')
    plt.ylabel('Actual Stars')
    plt.xlabel('Predicted Stars')
    plt.tight_layout()
    plt.savefig('confusion_matrix.png', dpi=300)
    plt.show()

print("\nEvaluation")


exact_accuracy = accuracy_calculaton(validation_preds, validation_labe
print(f"Exact accuracy: {exact_accuracy:.4f}")


plot_confusion_matrix(validation_preds, validation_labels)
```

```
Evaluation
Exact accuracy: 0.7033
```






```
def predict_star(review_text, model, tokenizer, device, max_length=128
  # now to use the model to predictthe
    model.eval()

    encoding = tokenizer(
        review_text,
        truncation=True,
        padding='max_length',
        max_length=max_length,
        return_tensors='pt'
    )

    input_ids = encoding['input_ids'].to(device)
    attention_mask = encoding['attention_mask'].to(device)

    with torch.no_grad():
        outputs = model(input_ids=input_ids, attention_mask=attention_
        _, prediction = torch.max(outputs.logits, dim=1)

    # adding 1 to show the rating from 1 - 5
    return prediction.item() + 1

# sample reviews
test_reviews = [
```

```
    "Shes a very nice person, but honestly, I cannot recommend her.
    "Professor Parsons is great about giving timely feedback, but does
    "Blended learning! Wednesdays usually just had a very simple discu
    "Really easy class and very laid-back professor. None of the non-r
    "Compared to other teachers for ENGL394 (from what I've heard fror
]

act_rating_reviews = [2,2,5,5,3]


# for i, (review, star) in enumerate(zip(test_reviews, act_rating_revi
#     prediction = predict_star(review, model, tokenizer, device)
#     print(f"\nReview {i+1}:")
#     print(f"Text: {review[:100]}...")
#     print(f"Predicted stars: {prediction}")
#     print(f"Actual stars: {star}")


correct = 0
prediction_details = []


for i, (review, actual_star) in enumerate(zip(test_reviews, act_rating
    predicted_star = predict_star(review, model, tokenizer, device)

    # exact match
    is_correct = (predicted_star == actual_star)
    if is_correct:
        correct += 1

    prediction_details.append({
        'review_num': i+1,
        'predicted': predicted_star,
        'actual': actual_star,
        'correct': is_correct,
        'error': abs(predicted_star - actual_star)  # How far off
    })

# calculatig acc
accuracy = correct / len(test_reviews) * 100
mean_absolute_error = sum(d['error'] for d in prediction_details) / le

# to display results

print(f"Total Reviews Tested: {len(test_reviews)}")
```

```
print(f"Correct Predictions: {correct}")
print(f"Accuracy: {accuracy:.1f}%")
print(f"Mean Absolute Error: {mean_absolute_error:.2f} stars")
print("="*70)


print("\nPREDICTION DETAILS:")

print(f"\n{'Review':<8} {'Predicted':<12} {'Actual':<10} {'Result':<1(

for detail in prediction_details:
    result = "CORRECT" if detail['correct'] else f"Off by {detail['er
    print(f"#{detail['review_num']:<7} {detail['predicted']}/5{'':<7}




from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
Total Reviews Tested: 5
Correct Predictions: 2
Accuracy: 40.0%
Mean Absolute Error: 0.80 stars
======================================================================

PREDICTION DETAILS:

Review   Predicted    Actual     Result
#1       1/5        2/5        Off by 1
#2       1/5        2/5        Off by 1
#3       5/5        5/5        CORRECT
#4       5/5        5/5        CORRECT
#5       1/5        3/5        Off by 2
```

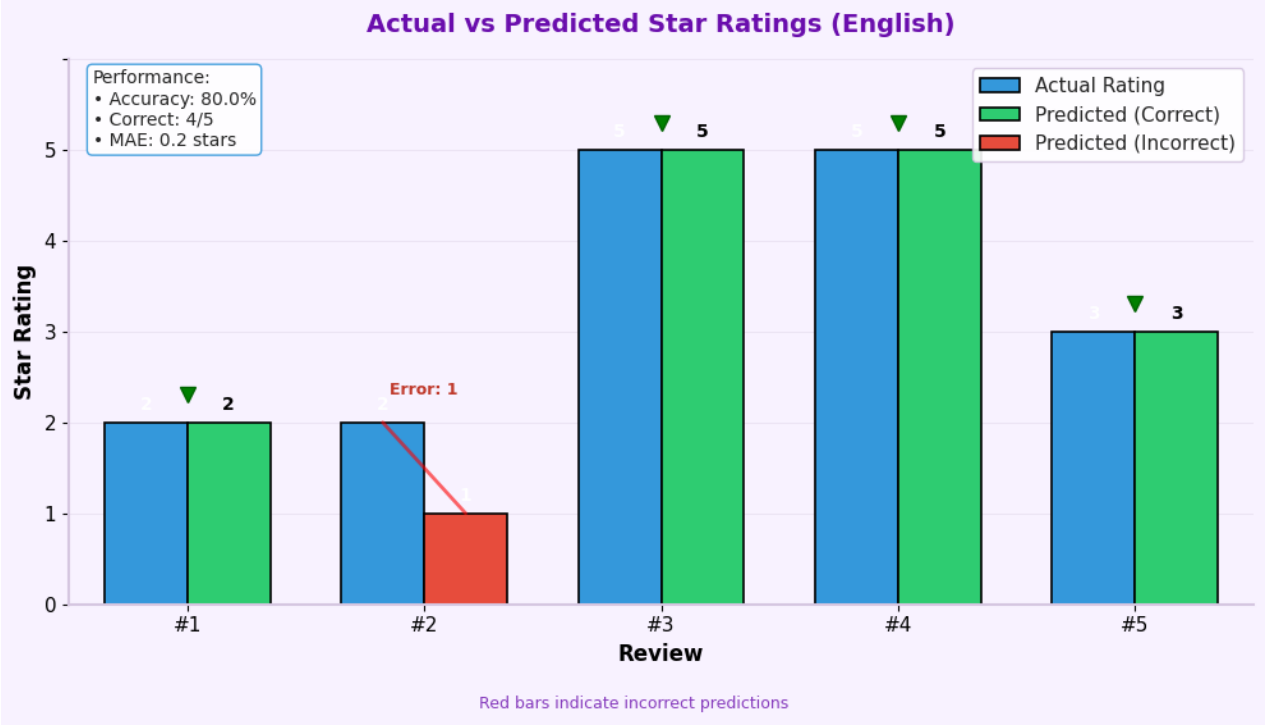
```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
from matplotlib.patches import Patch

# Your data
results_data = {
    'Review': ['#1', '#2', '#3', '#4', '#5'],
```

```
    'Actual': [2, 2, 5, 5, 3],
    'Predicted': [2, 1, 5, 5, 3],
    'Match': [True, False, True, True, True]
}

results = pd.DataFrame(results_data)

total_reviews = 5
correct_predictions = 4
accuracy = 80.0
mae = 0.20

# create the plot
plt.figure(figsize=(10, 6))
fig, ax = plt.subplots(figsize=(10, 6))

# purple background
fig.patch.set_facecolor('#f8f2ff')
ax.set_facecolor('#f8f2ff')
sns.set_style("ticks", {'axes.facecolor': '#f8f2ff'})

x = np.arange(len(results))
width = 0.35

bars1 = ax.bar(x - width/2, results['Actual'], width,
               label='Actual Rating', color='#3498db', edgecolor='bla

# plot predicted bars
for i, (pred, match) in enumerate(zip(results['Predicted'], results['M
    color = '#2ecc71' if match else '#e74c3c'
    label = None
    if i == 0 and match:
        label = 'Predicted (Correct)'
    elif i == 1 and not match:
        label = 'Predicted (Incorrect)'

    ax.bar(x[i] + width/2, pred, width,
           label=label, color=color, edgecolor='black', linewidth=1.2

ax.set_ylabel('Star Rating', fontsize=12, fontweight='bold')
ax.set_yticks([0, 1, 2, 3, 4, 5, 6])
ax.set_yticklabels(['0', '1', '2', '3', '4', '5', ''], fontsize=11)
ax.set_ylim(0, 6)
ax.set_xlim(-0.5, len(results) - 0.5)
```

```
    # x-axis
    ax.set_xlabel('Review', fontsize=12, fontweight='bold')
    ax.set_xticks(x)
    ax.set_xticklabels(results['Review'], fontsize=11)

    # title
    ax.set_title('Actual vs Predicted Star Ratings (English)',
                 fontsize=14, fontweight='bold', pad=15, color='#6a0dad')

    # add value labels on bars
    for i, (actual, pred, match) in enumerate(zip(results['Actual'], resu
        # actual value label
        ax.text(i - width/2, actual + 0.1, f'{actual}',
                ha='center', va='bottom', fontsize=10, fontweight='bold',

        # predicted value label
        text_color = 'white' if not match else 'black'
        ax.text(i + width/2, pred + 0.1, f'{pred}',
                ha='center', va='bottom', fontsize=10, fontweight='bold',

    # add result indicators
    for i, (actual, pred, match) in enumerate(zip(results['Actual'], resu
        y_pos = max(actual, pred) + 0.3
        if match:
            ax.plot(i, y_pos, 'gv', markersize=8, markeredgecolor='darkgr
        else:
            ax.text(i, y_pos, f"Error: {abs(pred-actual)}",
                    ha='center', fontsize=9, fontweight='bold', color='#C
            ax.plot([i - width/2, i + width/2], [actual, pred],
                    'r-', linewidth=2, alpha=0.6, zorder=1)

    # to create legend
    handles, labels = ax.get_legend_handles_labels()

    by_label = dict(zip(labels, handles))
    ax.legend(by_label.values(), by_label.keys(), fontsize=11, framealpha=
              frameon=True, edgecolor='#d5c6e0', facecolor='white',
              loc='upper right')

    # grid and styling
    ax.yaxis.grid(True, color='#e0d6e9', linestyle='-', linewidth=0.5, alp
    ax.set_axisbelow(True)
    sns.despine()

    # border
```

```
for spine in ax.spines.values():
    spine.set_color('#d5c6e0')
    spine.set_linewidth(1.5)

# add performance box
metrics_text = f'Performance:\n• Accuracy: {accuracy}%\n• Correct: {c
ax.text(0.02, 0.98, metrics_text, transform=ax.transAxes, fontsize=10
        verticalalignment='top', bbox=dict(boxstyle='round', facecolo
                                           edgecolor='#3498db', alpha

plt.tight_layout(rect=[0, 0.05, 1, 0.95])
plt.figtext(0.5, 0.02, 'Red bars indicate incorrect predictions',
            ha='center', fontsize=9, color='#6a0dad', alpha=0.8)

plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```



```
# Test with sample reviews
test_reviews = [
    "Do not take this professor if you have the option not to. She is
    "Professor Bardossy is a phenomenal teacher. I almost dropped thi
    "Padhi was extremely understanding and quick to respond to emails
    "THE MOST CRUEL MAN EVERRRRRRRRR truly it says something about yo
    "BEST PROFESSOR AT UMD. PERIOD. small workload, fun professor, he
    ]

act_rating_reviews = [1,5,4,1,5]


# for i, (review, star) in enumerate(zip(test_reviews, act_rating_rev
#     prediction = predict_star(review, model, tokenizer, device)
#     print(f"\nReview {i+1}:")
#     print(f"Text: {review[:100]}...")
#     print(f"Predicted stars: {prediction}")
#     print(f"Actual stars: {star}")


correct = 0
prediction_details = []

# to test each review
for i, (review, actual_star) in enumerate(zip(test_reviews, act_ratin
    predicted_star = predict_star(review, model, tokenizer, device)


    is_correct = (predicted_star == actual_star)
    if is_correct:
        correct += 1

    # storing details
    prediction_details.append({
        'review_num': i+1,
        'predicted': predicted_star,
        'actual': actual_star,
        'correct': is_correct,
        'error': abs(predicted_star - actual_star)  # How far off
    })
```

```
# calculate acc
accuracy = correct / len(test_reviews) * 100
mean_absolute_error = sum(d['error'] for d in prediction_details) / le

# to display results
print(f"Total Reviews Tested: {len(test_reviews)}")
print(f"Correct Predictions: {correct}")
print(f"Accuracy: {accuracy:.1f}%")
print(f"Mean Absolute Error: {mean_absolute_error:.2f} stars")


print("\nPREDICTION DETAILS:")

print(f"\n{'Review':<8} {'Predicted':<12} {'Actual':<10} {'Result':<1(

for detail in prediction_details:
    result = "CORRECT" if detail['correct'] else f"Off by {detail['er
    print(f"#{detail['review_num']:<7} {detail['predicted']}/5{'':<7}
```

```
Total Reviews Tested: 5
Correct Predictions: 4
Accuracy: 80.0%
Mean Absolute Error: 0.20 stars

PREDICTION DETAILS:

Review   Predicted    Actual     Result
#1       1/5        1/5        CORRECT
#2       5/5        5/5        CORRECT
#3       5/5        4/5        Off by 1
#4       1/5        1/5        CORRECT
#5       5/5        5/5        CORRECT
```

```
plt.figure(figsize=(9, 5))

# purple background
fig, ax = plt.subplots(figsize=(9, 5))
fig.patch.set_facecolor('#f8f2ff')  # light purple background
ax.set_facecolor('#f8f2ff')  # light purple background
```

```
sns.set_style("ticks", {'axes.facecolor': '#f8f2ff'})


x = np.arange(len(results))
width = 0.35

# plot actual ratings (blue bars)
bars1 = ax.bar(x - width/2, results['Actual'], width,
               label='Actual Rating', color='#3498db', edgecolor='black

correct_preds = []
incorrect_preds = []
correct_indices = []
incorrect_indices = []

for i, (pred, match) in enumerate(zip(results['Predicted'], results['Ma
    if match:
        correct_preds.append(pred)
        correct_indices.append(i)
    else:
        incorrect_preds.append(pred)
        incorrect_indices.append(i)

# plot correct predictions (green bars)
if correct_indices:
    ax.bar([x[i] + width/2 for i in correct_indices],
           correct_preds, width,
           label='Predicted (Correct)',
           color='#2ecc71', edgecolor='black', linewidth=1.2)

# plot incorrect predictions (red bars)
if incorrect_indices:
    ax.bar([x[i] + width/2 for i in incorrect_indices],
           incorrect_preds, width,
           label='Predicted (Incorrect)',
           color='#e74c3c', edgecolor='black', linewidth=1.2)

# rating labels
ax.set_ylabel('Star Ratings', fontsize=12, fontweight='bold')
ax.set_yticks([1, 2, 3, 4, 5])
ax.set_yticklabels(['1', '2', '3', '4', '5'], fontsize=11)
ax.set_ylim(0.5, 5.5)

# x-axis
ax.set_xlabel('Reviews', fontsize=12, fontweight='bold')
```

```
ax.set_xticks(x)
ax.set_xticklabels(results['Review'], fontsize=11)

# title
ax.set_title('Actual vs Predicted Star Ratings (Business)',
             fontsize=14, fontweight='bold', pad=15, color='#6a0dad')

# add result
for i, (actual, pred, match) in enumerate(zip(results['Actual'], result
    y_pos = max(actual, pred) + 0.15
    if match:
        # green for correct predictions
        ax.plot(i, y_pos, 'gv', markersize=8, markeredgecolor='darkgree
    else:
        # red error magnitude
        ax.text(i, y_pos, f"✗ {abs(pred-actual)}",
                ha='center', fontsize=10, fontweight='bold', color='#c0
        # red error line
        ax.plot([i - width/2, i + width/2], [actual, pred],
                'r-', linewidth=2, alpha=0.6, zorder=1)

from matplotlib.patches import Patch

# create legend patches for all three categories
legend_patches = [
    Patch(facecolor='#3498db', edgecolor='black', label='Actual Rating'
    Patch(facecolor='#2ecc71', edgecolor='black', label='Predicted (Cor
    Patch(facecolor='#e74c3c', edgecolor='black', label='Predicted (Inc
]

# create legend
ax.legend(handles=legend_patches, fontsize=11, framealpha=0.75,
          frameon=True, edgecolor='#d5c6e0', facecolor='white',
          loc='upper left')


sns.despine()

ax.yaxis.grid(True, color='#e0d6e9', linestyle='-', linewidth=0.5, alph
ax.set_axisbelow(True)

for spine in ax.spines.values():
    spine.set_color('#d5c6e0')
    spine.set_linewidth(1.5)
```

```
plt.tight_layout(rect=[0, 0.05, 1, 0.95])

plt.figtext(0.5, 0.02, 'Red bars indicate incorrect predictions',
            ha='center', fontsize=9, color='#6a0dad', alpha=0.8)

plt.show()
```

```
<Figure size 900x500 with 0 Axes>
```